

Keywords: mathematical reasoning • LLM interpretability
• structural understanding • primitive-guided post-training

The Missing Primitive: Diagnosing and Repairing Mathematical Reasoning in Large Language Models

A New Benchmark Reveals that Discovery—Not Execution—Is the Universal Bottleneck to LLM Mathematical Reasoning, and That Targeting It Yields Consistent Gains

By Shuo Xing, Zilin Dai, Chengyuan Qian, Fangzhou Lin, Wenjing Chen, Ping He, Pan Lu, Alvaro Velasquez, Mohit Bansal & Zhengzhong Tu (Texas A&M University; UNC Chapel Hill; UCLA; University of Colorado Denver)

arXiv:2610.02191 • October 2026

LLMs that routinely solve competition-level mathematics often lack the structural understanding underlying their solutions: given the correct foundational idea—a *Mathematical Primitive*—the same models execute near-perfectly, but independently discovering that primitive is the dominant bottleneck across all frontier architectures tested. Primitive-guided post-training closes the gap.

AT A GLANCE

Problem: LLM mathematical reasoning accuracy does not reveal whether models possess the structural understanding behind their solutions.

Why it matters: Models with identical benchmark scores may rely on brittle heuristics; diagnosing structural gaps enables targeted improvement.

Method: Mathematical Primitive concept plus the Prim benchmark spanning Discovery, Generation, Digestion, and Execution dimensions.

Result: Discovery lags Execution by up to 30 pp; primitive-guided post-training consistently raises both Discovery and overall task accuracy.

Evidence: Evaluated across multiple frontier LLMs on Prim and held-out downstream math benchmarks.

The Big Picture

Large language models have crossed remarkable thresholds in mathematical reasoning: the best contemporary systems solve a substantial fraction of AMC and AIME problems and approach expert performance on graduate-level benchmarks. Yet this aggregate success conceals a potentially critical gap: does solving a problem indicate that the model grasped the foundational idea organizing

the solution, or merely that it reproduced a pattern from training data?

This paper takes a first step toward answering that question by introducing the concept of the *Mathematical Primitive*—a concise, non-procedural description of the core insight that, once identified, makes the detailed execution steps straightforward. For example, a combinatorics problem’s primitive might be “map to a stars-and-bars counting argument” rather than the specific arithmetic that follows from it.

Why Existing Approaches Fall Short

Benchmark saturation: State-of-the-art LLMs now score near ceiling on MATH and AMC; high scores mask vastly different capability profiles.

No structural audit: Existing benchmarks measure correct final answers, not whether the model identified the organizing idea. Two models with identical accuracy may differ dramatically in structural understanding.

Post-training targeting: Standard chain-of-thought fine-tuning and process reward models treat all reasoning steps equally; they do not specifically target primitive discovery where diagnosis reveals the greatest weakness.

Core Mechanism

Mathematical Primitive. A primitive is defined as the single, concise, non-procedural representation of the foundational idea that organizes a solution. It is shorter than a proof sketch, more specific than a problem category, and must identify the key conceptual move.

Prim Benchmark. The paper constructs Prim with four complementary dimensions:

- *Discovery:* Given the problem, identify the primitive.
- *Generation:* Given the primitive, produce a valid solution.
- *Digestion:* Given a complete solution, extract the primitive it relies on.
- *Execution:* Given both problem and primitive, carry out the solution steps.

Technical Formulation

Let $\mathcal{P}$ be the space of primitives and $\mathcal{Q}$ the space of problems. Define four binary capability indicators for a model $\mathcal{M}$ on problem q with ground-truth primitive $p^*(q)$:

$$\text{Disc}(q) = \mathbf{1}[\hat{p}(q) = p^*(q)]$$

$$\text{Exec}(q, p^*) = \mathbf{1}[\hat{s}(q, p^*(q)) \text{ is correct}]$$

The central finding is:

$$\mathbb{E}_q[\text{Exec}(q, p^*(q))] \gg \mathbb{E}_q[\text{Disc}(q)]$$

for all tested models, establishing that **Discovery is the dominant bottleneck**. Primitive-guided post-training minimizes a combined loss:

$$\mathcal{L} = \mathcal{L}_{\text{solve}} + \lambda \mathcal{L}_{\text{discover}}$$

where $\mathcal{L}_{\text{discover}}$ is a cross-entropy term on primitive Discovery, and λ is a balancing hyperparameter.

Learning or Inference Procedure

Evaluation:

1. Run each Prim dimension independently on the same model.
2. Compute per-dimension accuracy and capability profile.
3. Compare profiles across models with similar aggregate task accuracy.

Post-training:

1. Annotate training problems with ground-truth or LLM-generated primitives.
2. Add a primitive Discovery objective alongside the standard solution objective.
3. Fine-tune with combined loss $\mathcal{L}$; evaluate on held-out Prim dimensions and downstream benchmarks.

What the Guarantee Says

No formal convergence bound is given; the Discovery bottleneck is an empirical finding. The central claim is: across all tested frontier LLMs, Execution given a correct primitive substantially exceeds unconditioned Discovery, establishing the latter as universally limiting. Post-training improvement transfers to held-out benchmarks not seen during fine-tuning.

Experimental Findings

- **Discovery gap:** Execution given the correct primitive exceeds unconditioned Discovery by up to 30 percentage points across all frontier LLMs.
- **Hidden differences:** Two models with identical MATH accuracy can differ by over 20 pp on the Discovery dimension alone.
- **Latent execution capacity:** Providing correct primitives to models that cannot discover them independently unlocks near-full solution accuracy.
- **Post-training gains:** Primitive-guided fine-tuning consistently improves Discovery accuracy and overall task performance, with gains transferring to held-out sets.

Ablations and Interpretation

Primitive granularity: Coarser primitives yield lower Execution improvement; fine-grained primitives that specify the exact conceptual move are most effective.

Imperfect primitives: Using LLM-generated rather than ground-truth primitives reduces but does not eliminate the Execution advantage—even imperfect discovery guidance helps.

Digestion vs. Discovery: High Digestion accuracy does not predict high Discovery accuracy, establishing that recognizing a primitive in a given solution and independently finding it from a problem statement are distinct capabilities.

Reference

Shuo Xing et al. **The Missing Primitive: Diagnosing and Repairing Mathematical Reasoning in Large Language Models**. arXiv:2610.02191, October 2026.

<https://arxiv.org/abs/2610.02191>